

SAPPHIRE CODING AND USAGE GUIDE

Sapphire is an autonomous-first programming language with built-in ML, AI, distributed training, and PC automation. This guide covers the complete syntax and standard library API.

1. Basic Syntax

```
let name = "Sapphire"; let count = 42;
print("Hello {name}!");
fn greet(x) { print("Hi {x}!"); }
greet(name);
if (count > 10) { print("Big"); } else { print("Small"); }
for (i in [1,2,3]) { print(i); }
```

2. Colorless Concurrency

```
parallel {
  { let a = os.system_info(); print(a.cpu_percent); }
  { let b = http.get("https://api.example.com"); }
  { let c = fs.read("log.txt"); }
}
```

3. ML and Deep Learning Stack

Capability	Sapphire API
Tensors	ml.tensor(), .add(), .matmul(), .reshape(), .relu(), .softmax()
Autograd	ml.autograd.variable(), .gradient(), .tape()
Datasets	ml.dataset.from_csv(), .random(), .batch(), .normalize(), .split()
Models	ml.model.linear(), .relu(), .dropout(), .batch_norm(), .sequential(), .mlp()
Training	ml.train.fit(model, dataset, loss, optim, epochs=N, batch_size=B)
Kernels	ml.kernel.gemm(), .conv2d(), .max_pool2d(), .layer_norm(), .fft()
GPU/TPU	ml.gpu.info(), .to_device(), .synchronize(), .memory_stats()
Optimizers	ml.optim.adam(), .sgd(), .rmsprop()
Loss	ml.loss.mse(), .cross_entropy(), .binary_cross_entropy(), .mae()
Frontier LLM	ml.distributed.Transformer(), .Cluster(), .train() -- 5D frontier

4. Frontier Distributed LLM (ml.distributed) -- NEW in v1.0.0

```
// 70B model on 512x H100-80GB
let model = ml.distributed.Transformer({"layers": 80, "hidden": 8192,
  "heads": 64, "vocab": 128000, "seq_len": 8192, "precision": "fp8"});
let cluster = ml.distributed.Cluster({"gpu_type": "H100-80GB",
  "num_gpus": 512, "gpus_per_node": 8});
let result = ml.distributed.train(model, cluster,
  {"tokens": 1000000000000, "batch_size": 2048, "checkpoint_every": 1000});
print("Strategy: {result.strategy}"); // TP1_PP1_DP512_ZeR03_FP8
print("MFU: {result.mfu_percent}%"); // 61.0%
print("ETA: {result.eta_days} days"); // ~76 days for 10T tokens
```

5. Frontier LLM Parallelism Capabilities

Capability	Detail
TP (Tensor Parallelism)	Splits attention heads/MLP columns across GPUs within node. Uses NVLink.
PP (Pipeline Parallelism)	Layer micro-batching across pipeline stages (1F1B schedule).

SAPPHIRE v1.0.0 | Sapphire Coding and Usage Guide

DP (Data Parallelism)	Gradient AllReduce across DP ranks after each backward pass.
EP (Expert Parallelism)	MoE router + expert sharding across EP ranks.
SP (Sequence Parallelism)	Sequence-dimension split for ultra-long context.
ZeRO-1/2/3 / FSDP	Optimizer, gradient, and weight sharding across DP ranks.
5D Auto-Parallelism Solver	Exhaustive TP x PP x DP x EP x SP grid search -- auto picks best strategy.
FlashAttention-3	IO-optimal fused attention kernel for H100/H200. SRAM-only.
FP8 TransformerEngine	FP8 matmuls with BF16 accumulation. ~2x throughput vs BF16.
NCCL AllReduce/AllGather	Topology-aware ring + recursive halving collective scheduling.
PyTorch FSDP Codegen	Auto-emit production training scripts + SLURM launcher.
Fault Tolerance	Checkpoint-every-N-steps, resume on node failure.